

Operating Systems & System Hardening

The Critical 20% That Unlocks 80% of Understanding

A Comprehensive Guide for Security Engineers

Linux & Windows • Process Security • File Permissions • Attack Surface Reduction

Introduction: Why Operating System Knowledge is Non-Negotiable

As a security engineer, you live at the intersection of systems and security. Every application runs on an operating system. Every attack exploits OS features or misconfigurations. Understanding how Linux and Windows work—not just using them, but understanding their internals—is the difference between reactive patching and proactive defense.

This guide identifies the **critical 20% of OS concepts** that enable you to understand the remaining 80% of system security. These are the concepts that appear in every breach analysis, every hardening guide, and every security engineering interview.

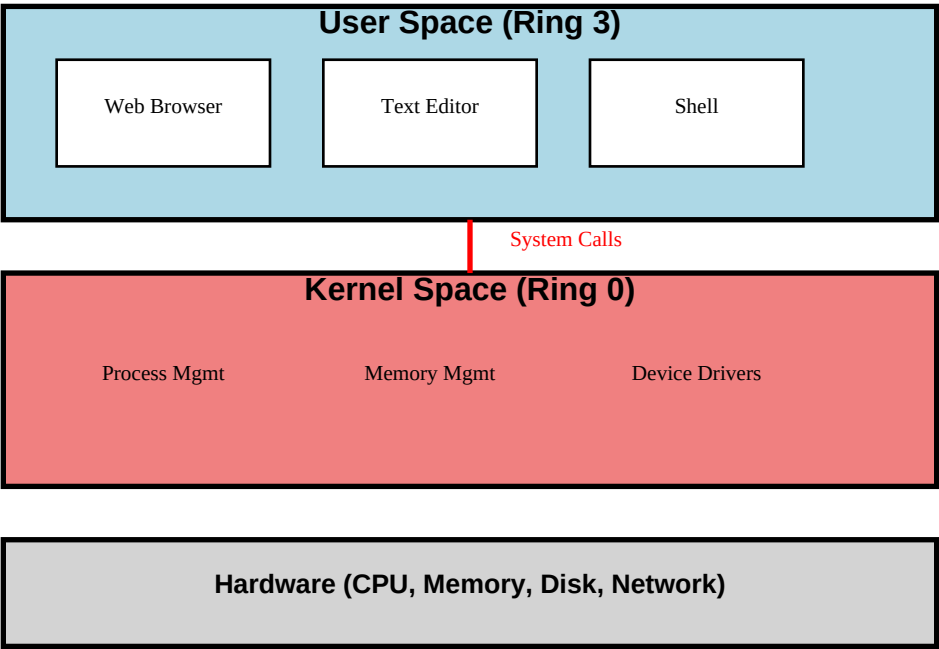
Concept	Why It's Critical for Security
Process Management & Privilege Model	Understanding privilege escalation, process injection, and containment
File System Permissions & Access Control	Most attacks exploit misconfigured permissions; critical for defense
Service Management & Attack Surface	Every running service is a potential entry point; reduction is key
Authentication Mechanisms & Account Security	Credential theft is the #1 initial access vector; must understand deeply

Each concept includes both Linux and Windows perspectives, as modern security engineers must be bilingual in these environments. We'll explore what attackers target, how to harden systems, and—critically—the nuances that don't appear in standard documentation.

CONCEPT 1: Process Management & Privilege Model

Understanding How Programs Execute and Escalate

The Analogy: Think of processes like employees in a company. Each has specific permissions (clearance levels), operates in a defined space (memory), and can request resources or communicate with others. The OS kernel is like HR and security combined—it enforces who can do what, mediates all resource access, and prevents chaos.



The Privilege Separation Model

Modern operating systems separate user space (unprivileged) from kernel space (privileged) to protect system integrity. This separation is fundamental to security:

Layer	Description	Security Implications
User Space (Ring 3)	Where applications run. Limited access to hardware. Must request OS services via system calls.	Most exploits start here. Goal is to escape to kernel.
Kernel Space (Ring 0)	OS kernel with full hardware access. Manages all processes, memory, devices.	Escapes, control. Kernel exploits = full system compromise.
System Calls	Interface between user and kernel. Controlled entry for privileged operations (open file, create process).	Attack surface. Malformed syscalls can exploit kernel bugs.

Process Attributes: The Security Context

Every process has attributes that define what it can do. Understanding these is critical for detecting privilege escalation:

Linux Process Attributes:

Attribute	Description	Security Relevance
UID/EUID	Real and Effective User ID	EUID determines actual permissions. SUID binaries run with owner's EUID.
GID/EGID	Real and Effective Group ID	Group-based permissions. Supplementary groups also apply.
Capabilities	Fine-grained privileges (CAP_NET_RAW, CAP_SYS_ADMIN, etc.)	Can bypass root restrictions. CAP_SYS_ADMIN is all-or-nothing root. Can be dangerous.
Process ID (PID)	Unique process identifier	PID 1 is init/systemd. Low PIDs often system processes.
Parent PID (PPID)	Process that spawned this one	Tracks process ancestry. Orphans get adopted by PID 1.
Session/Terminal	Associated login session	Background processes detach. Daemons have no terminal.

Windows Process Attributes:

Attribute	Description	Security Relevance
Access Token	Security context with SID, privileges, groups	Defines what process can access. Token theft is common attack.
Integrity Level	Low, Medium, High, System (Vista+)	Prevents low-privilege processes from accessing high-privilege objects.
Process ID (PID)	Unique identifier	Similar to Linux. System processes have low PIDs.
Session ID	Terminal services session	Session 0 is for services. Interactive users in session 1+.
Image Name	Executable path	svchost.exe, lsass.exe, etc. Attackers masquerade using similar names.

Real-World Scenario: SUID Privilege Escalation

Scenario: You find a vulnerable web application on a Linux server. The application runs as www-data (low privilege). During enumeration, you discover /usr/bin/find has SUID bit set and is owned by root.

Analysis: The SUID (Set User ID) bit means this binary runs with the owner's privileges (root) regardless of who executes it. The 'find' command can execute other commands via -exec flag. An attacker can exploit this:

```
$ find /etc/passwd -exec /bin/sh \;
```

This spawns a shell running as root (EUID=0), giving full system control.

Defense: Audit all SUID binaries regularly. Only essential system binaries should have SUID. Find them with:

```
$ find / -perm -4000 -type f 2>/dev/null
```

Command-Line Process Investigation

Linux Process Analysis:

```
$ ps aux | grep -v grep
```

Lists all processes with user, CPU, memory, command. Look for suspicious names or high resource usage.

```
$ ps -eo pid,user,euser,ruser,cmd
```

Shows real vs effective user. Mismatches indicate SUID execution or privilege changes.

```
$ lsof -p [PID]
```

Lists open files, network connections, and resources for a process. Critical for incident analysis.

```
$ pstree -p
```

Displays process tree with PIDs. Useful for tracking parent-child relationships and finding orphaned processes.

Windows Process Analysis:

```
C:\> tasklist /v
```

Lists processes with PID, session, user, memory. Look for unusual process names or users.

```
C:\> wmic process get processid,parentprocessid,name,executablepath
```

Shows process lineage and executable paths. Attackers hide malware in temp directories.

```
C:\> Get-Process | Select-Object Name,Id,Path,Company | Where-Object {$_.Company -eq $null}
```

PowerShell to find processes without valid digital signatures. Unsigned = suspicious.

```
C:\> netstat -anob
```

Shows network connections with associated process. Identifies C2 communications.

Interview Question Pattern

Common question: "You see a process running as root/SYSTEM that you don't recognize. How do you investigate?"

Systematic approach:

1. Identify: Note PID, PPID, command line, executable path
2. Context: When did it start? From where? Parent process?
3. Network: Open connections? Listening ports?
4. Files: What files is it accessing? Any unusual patterns?
5. Persistence: Is it started at boot? Service? Cron job?
6. Signature: Signed binary? Hash match known malware?
7. Correlate: Check logs, firewall, IDS for related activity

■ Under-Explored Perspectives: Process Management

1. Process Injection Techniques: Attackers inject code into legitimate processes to evade detection (DLL injection, process hollowing, reflective loading). Understanding memory layout and how processes load libraries is rarely taught but critical for detecting advanced threats.

2. Zombie and Orphan Processes: A zombie is terminated but not reaped by parent (shows as `Z`). An orphan's parent died, so it's adopted by `init` (PID 1). Both can indicate malware or exploitation, but this is often overlooked in process monitoring.

3. Linux Capabilities vs. Traditional Root: Many think root is all-or-nothing, but Linux capabilities split root privileges into 40+ granular permissions. A process with `CAP_NET_RAW` can craft packets; `CAP_SYS_ADMIN` is nearly full root. Understanding capabilities helps with privilege minimization.

4. Windows Token Manipulation: Attackers steal access tokens from high-privilege processes (`lsass.exe`, `winlogon.exe`) and impersonate them. Tools like `Mimikatz` abuse this. Yet understanding how Windows uses tokens for access control is often superficial.

5. Fork Bombs and Resource Exhaustion: A fork bomb creates processes recursively until system resources are exhausted. Understanding process limits (`ulimit` on Linux, job objects on Windows) and detection is important but rarely covered.

CONCEPT 2: File System Permissions & Access Control

The Foundation of OS Security

The Analogy: File permissions are like locks on doors in a building. Some doors are open to everyone (world-readable), some only to specific departments (group-readable), and some only to the owner (owner-only). The OS enforces these locks, preventing unauthorized access.

```
/etc/shadow  
-rw-r----- 1 root shadow 1234 Jan 1 12:00
```

Owner: root
rw- (6)

Group: shadow
r-- (4)

Others: all
--- (0)

r=read(4) w=write(2) x=execute(1)

Linux File Permissions: The rwx Model

Linux uses a straightforward permission model: read (r), write (w), execute (x) for three categories: owner, group, others. This is represented in octal notation (0-7) where each permission has a value:

Permission	Symbol	Octal Value	Meaning
Read	r	4	View file contents or list directory
Write	w	2	Modify file or create/delete files in directory
Execute	x	1	Run file as program or enter directory
None	-	0	No permission

Common Permission Patterns

Octal	Symbolic	Meaning	Use Case
644	-rw-r--r--	Owner: rw, Group: r, Others: r	Public readable files, config files
600	-rw-----	Owner: rw, Group: none, Others: none	Private files (SSH keys, passwords)
755	-rwxr-xr-x	Owner: rwx, Group: rx, Others: rx	Executable scripts, directories
700	-rwx-----	Owner: rwx, Group: none, Others: none	Private directories, sensitive scripts

777	-rwxrwxrwx	Everyone: full access	DANGEROUS! Avoid except temp dirs
000	-----	No one: no access	Locked files (even owner blocked)

Special Permission Bits

Beyond basic rwx, Linux has special permission bits that modify behavior:

Bit	Name	Octal	Effect on Files	Effect on Directories	Security Risk
s (user)	SUID	4000	Execute as file owner	No effect	HIGH: Can escalate privileges
s (group)	SGID	2000	Execute as file group	New files inherit directory group	MEDIUM: Group-based escalation
t	Sticky	1000	Prevents deletion by others	Only owner can delete own files	LOW: Protects /tmp

Windows File Permissions: ACLs

Windows uses Access Control Lists (ACLs) which are far more granular than Linux's basic model. Each file has a Discretionary Access Control List (DACL) containing Access Control Entries (ACEs) that specify what each user/group can do:

Permission	Description	Common Use
Full Control	Complete access: read, write, delete, change permissions	System administrators
Modify	Read, write, delete (but not change permissions)	Standard users on their documents
Read & Execute	View and run files	Program Files, system directories
Read	View only	Public shared files
Write	Create and modify (but not read!)	Drop boxes, rarely used alone
Special Permissions	Granular control (take ownership, change ACLs)	Advanced scenarios

Real-World Scenario: World-Writable Configuration File

Scenario: During a security audit, you find `/etc/cron.d/backup` has permissions `666 (rw-rw-rw-)`. This file contains a cron job that runs as root.

Analysis: ANY user can modify this file. An attacker with limited access can add malicious commands to the cron job, which will execute as root:

```
$ echo '* * * * * root /tmp/malware.sh' >> /etc/cron.d/backup
```

This achieves privilege escalation from any user to root.

Defense: System configuration files must be owned by root with permissions `644` or stricter. Audit with:

```
$ find /etc -type f -perm -002 2>/dev/null
```

This finds all world-writable files in `/etc`.

Command-Line Permission Management

Linux Commands:

```
$ ls -la /path/
Lists files with permissions, owner, group, size, timestamp. The 'l' shows details, 'a' shows hidden files.

$ chmod 600 /home/user/.ssh/id_rsa
Sets file to owner-only read/write. Critical for SSH private keys.

$ chown root:root /etc/passwd
Changes owner to root, group to root. Ensures system files aren't user-modifiable.

$ getfacl /path/file
Shows extended ACLs. Linux supports POSIX ACLs beyond basic rwx.

$ find / -type f \( -perm -4000 -o -perm -2000 \) 2>/dev/null
Finds all SUID and SGID binaries. These are privilege escalation targets.
```

Windows Commands:

```
C:\> icacls C:\Path\File.txt
Displays ACL. Shows each user/group and their permissions.

C:\> icacls C:\Path /grant Users:(R) /T
Grants 'Users' group Read permission recursively. Use for bulk permission changes.

C:\> takeown /F C:\Path\File.txt
Takes ownership of file. Requires admin rights. Used to recover from permission lockouts.

C:\> Get-Acl C:\Path | Format-List
PowerShell to view ACL in readable format. Shows inherited vs explicit permissions.
```

Interview Question Pattern

Common question: "A user can read a file but claims they can't modify it, even though they're the owner. Why?"

Analysis: Check:

1. Effective permissions: Use `getfacl` (Linux) or `icacls` (Windows) to see actual permissions
2. Read-only attribute: File might be marked read-only (`chattr +i` on Linux, `attrib +R` on Windows)
3. Filesystem mounted read-only: Mount options may prevent all writes
4. SELinux/AppArmor: MAC policies may deny write even if DAC allows
5. File locks: Another process may have exclusive lock
6. Immutable flag: `chattr +i` makes file unchangeable even by root

■ Under-Explored Perspectives: File Permissions

1. Directory Execute Permission: Many don't realize execute (x) on a directory means 'can enter directory,' not 'can run'. Without it, you can't cd into the directory or access its files, even if they're world-readable. This is often the source of 'Permission denied' errors.

2. Permission Inheritance vs. Default Permissions: On Windows, permissions inherit from parent by default. Breaking inheritance is a common misconfiguration. On Linux, umask determines default permissions for new files, but it's often misunderstood.

3. The Root of All Evil: Root (UID 0) bypasses all file permission checks. Even files with 000 permissions are accessible to root. The only exception is execute—root needs at least one x bit set somewhere. This is why least privilege is critical.

4. NTFS Alternate Data Streams: Windows files can have hidden streams (file.txt:hidden) with separate permissions. Malware hides in ADS. Standard tools like 'dir' don't show them. Use 'dir /R' or streams.exe to detect.

5. Linux Extended Attributes and Capabilities: Beyond standard permissions, files can have extended attributes (xattr) and file capabilities. A binary with cap_net_raw=ep can create raw sockets without being SUID root. This is powerful but rarely audited.

CONCEPT 3: Service Management & Attack Surface Reduction

Every Running Service is a Potential Entry Point

The Principle: Attack surface is the sum of all points where an attacker can try to enter or extract data. Every running service—web server, SSH, database—is a door into your system. Reducing attack surface means closing unnecessary doors.

Understanding Services

Services (daemons on Linux, services on Windows) are background processes that start automatically and run without user interaction. They provide functionality like web serving, remote access, time synchronization, and more.

Aspect	Linux	Windows
Init System	systemd (modern), SysV init (legacy)	Service Control Manager (SCM)
Service Files	/etc/systemd/system/*.service	Registry: HKLM\SYSTEM\CurrentControlSet\Services
Management CLI	systemctl, service	sc.exe, net.exe
Service User	Configurable per service	LocalService, NetworkService, LocalSystem
Auto-start	enabled/disabled	Automatic/Manual/Disabled
Dependencies	After=, Requires=	DependOnService

High-Risk Services to Monitor

Service	Port	Risk	Why It Matters
SSH (sshd)	22/tcp	HIGH	Remote access. Brute-force target. Credential theft.
RDP (Terminal Services)	3389/tcp	HIGH	Windows remote desktop. Heavily exploited. BlueKeep.
Telnet	23/tcp	CRITICAL	Unencrypted remote access. Never use in production.
FTP (vsftpd, IIS FTP)	21/tcp	HIGH	Unencrypted file transfer. Credentials in clear text.
SMB (Samba, Windows)	445/tcp	CRITICAL	File sharing. WannaCry, EternalBlue, ransomware vector.
Web servers (Apache, nginx, IIS)	80, 443	HIGH	Application vulnerabilities. SQL injection, XSS.
Database (MySQL, PostgreSQL, Oracle)	3306, 5432, 1521	HIGH	Direct DB access. Data exfiltration.
DNS (named, dnsmasq)	53/udp	MEDIUM	Cache poisoning, amplification attacks.

Real-World Scenario: Unnecessary Service Exploitation

Scenario: A web server was compromised via SQL injection. The attacker established persistence by installing a backdoor service that runs on boot. The service wasn't detected because it was named 'systemd-updates' to blend in.

Analysis: The attacker created a service that starts automatically, listens on a non-standard port, and provides remote shell access. Because system administrators didn't have a baseline of legitimate services, the malicious one went unnoticed.

Defense Strategy:

1. **Baseline:** Document all legitimate services and their purposes
2. **Disable unnecessary services:** If it's not needed, turn it off
3. **Monitor for changes:** Alert on new services or modifications
4. **Least privilege:** Services should run as dedicated low-privilege accounts
5. **Regular audits:** Compare running services against baseline

Command-Line Service Management

Linux (systemd):

```
$ systemctl list-units --type=service --state=running
Lists all running services. Look for unexpected entries.

$ systemctl status sshd
Shows service status, PID, memory usage, recent logs. Check if service should be running.

$ systemctl disable telnet.service
Prevents service from starting at boot. Use for risky services you might need occasionally.

$ systemctl stop apache2.service
Stops service immediately. Doesn't prevent restart on boot if enabled.

$ journalctl -u sshd -n 100
Shows last 100 log entries for SSH service. Look for failed login attempts.
```

Windows:

```
C:\> sc query type= service state= all
Lists all services and their states. Note the space after '=' is required.

C:\> sc qc [service_name]
Shows service configuration: binary path, startup type, account.

C:\> Get-Service | Where-Object {$_.Status -eq 'Running'}
PowerShell to list running services. More flexible filtering.

C:\> Set-Service -Name Telnet -StartupType Disabled
Disables service from auto-starting. PowerShell method.

C:\> Get-WinEvent -LogName System | Where-Object {$_.Id -eq 7045}
Event ID 7045 = new service installed. Critical for detecting persistence.
```

Hardening Best Practices

Action	Linux	Windows	Impact
Disable unnecessary services	systemctl disable [service]	sc config [service] start=disabled	Reduces attack surface
Run services as non-root/systemd user	user= in service file	Use managed service accounts	Limits damage from compromise
Restrict network access	Firewall rules (iptables, ufw)	Windows Firewall rules	Prevents external exploitation
Enable logging	journald, rsyslog	Event Log, Sysmon	Detection and forensics
Regular updates	apt update && apt upgrade	Windows Update, WSUS	Patches vulnerabilities
File integrity monitoring	AIDE, Tripwire	FSRM, third-party tools	Detects unauthorized changes

Interview Question Pattern

Common question: "How would you reduce the attack surface on a newly deployed Linux web server?"

Systematic approach:

1. Audit running services: `systemctl list-units --type=service`
2. Disable unnecessary: Stop and disable services like cups (printing), bluetooth, avahi (mDNS)
3. Secure necessary services: Configure SSH (disable root login, key-only auth), restrict web server
4. Firewall: Only allow ports 80, 443, 22 (from specific IPs)
5. Updates: Ensure automatic security updates enabled
6. Monitoring: Enable auditd, configure log forwarding to SIEM
7. File permissions: Ensure web files owned by web user, not writable by world
8. SELinux/AppArmor: Enforce mandatory access control policies

■ Under-Explored Perspectives: Service Management

1. Service Dependencies and Cascading Failures: Services often depend on others. Disabling a service might break dependent services. Understanding systemd's 'Wants=' vs 'Requires=' directives helps prevent outages, but this is rarely taught.

2. Socket Activation: Systemd can start services on-demand when a port is accessed (socket activation). The service isn't 'running' normally but activates when needed. This can hide services from simple 'systemctl list' checks.

3. Windows Service Recovery Actions: Windows services can be configured to restart automatically on failure, run a program, or reboot the system. Attackers abuse this for persistence—if their malware service crashes, it auto-restarts.

4. The Danger of 'Disabled' Services: A disabled service doesn't auto-start at boot, but it can still be started manually or by another service. True security requires removing the binary or strict firewall rules.

5. Containerized Services and Init Systems: In containers, PID 1 must reap zombie processes. Many containers run applications directly as PID 1, which doesn't handle signals properly. This can prevent graceful shutdowns and cause security tools to miss process terminations.

CONCEPT 4: Authentication Mechanisms & Account Security

The First Line of Defense

The Reality: Credential theft and abuse is the #1 initial access vector in breaches. Understanding authentication mechanisms—how users prove their identity—is fundamental to preventing unauthorized access.

Authentication Factors

Factor	Type	Examples	Weakness
Something you know	Knowledge	Passwords, PINs, security questions	Can be guessed, phished, or stolen
Something you have	Possession	Smart cards, hardware tokens, phone apps	Can be lost or stolen
Something you are	Inherence	Fingerprint, facial recognition, iris scan	Can't be changed if compromised
Somewhere you are	Location	GPS location, IP address range	Can be spoofed or bypassed

Multi-Factor Authentication (MFA) combines two or more independent factors. Even if one factor is compromised (e.g., password phished), the attacker still needs the second factor. This dramatically reduces unauthorized access.

Linux Authentication Stack

Linux uses PAM (Pluggable Authentication Modules) for flexible authentication. PAM separates authentication from applications, allowing centralized policy:

Component	Purpose	Configuration	Security Use
pam_unix.so	Traditional /etc/passwd, /etc/shadow auth	/etc/pam.d/common-auth	Basic password authentication
pam_ldap.so	LDAP/Active Directory authentication	/etc/pam.d/common-auth	Centralized authentication
pam_faillock.so	Account lockout after failed attempts	/etc/pam.d/common-auth	Brute-force protection
pam_pwquality.so	Password complexity requirements	/etc/pam.d/common-password	Enforce strong passwords
pam_google_authenticator.so	TOTP-based MFA	/etc/pam.d/sshd	Two-factor authentication

Windows Authentication Mechanisms

Method	Description	Security Notes
NTLM	Legacy challenge-response protocol	Vulnerable to pass-the-hash, relay attacks. Disable if possible.
Kerberos	Ticket-based authentication (domain)	More secure than NTLM. Tickets can still be stolen (Golden Ticket).
Smart Card	Certificate-based authentication	Strong authentication. Requires PKI infrastructure.
Windows Hello	Biometric/PIN authentication	Modern, passwordless. Tied to device TPM.
Local Accounts	Stored in SAM database	Hashes stored in SYSTEM registry hive. Can be extracted.

Password Storage Security

Systems must never store plaintext passwords. Instead, they store cryptographic hashes. Understanding how this works is critical for security:

System	Storage	Algorithm	Vulnerability
Linux (modern)	/etc/shadow	SHA-512, bcrypt, or Argon2 with salt	Relatively secure with strong passwords
Linux (legacy)	/etc/passwd	DES crypt (weak)	Easily cracked. Should never be used.
Windows (local)	SAM (Security Account Manager)	NTLM hash (MD4-based)	No salt. Vulnerable to rainbow tables, pass-the-hash.
Windows (domain)	Active Directory database	Kerberos (AES) for auth, NTLM stored	NTLM hashes still extractable. Domain admin compromise

Real-World Scenario: Pass-the-Hash Attack

Scenario: An attacker compromises a Windows workstation via phishing. They extract NTLM password hashes from memory using Mimikatz. With the hash of a domain admin account, they authenticate to other systems *without needing the plaintext password*.

Analysis: Windows NTLM authentication accepts the hash directly for authentication (pass-the-hash). The attacker doesn't need to crack the password—the hash itself is the credential. This allows lateral movement across the domain.

Defense:

- 1. Disable NTLM where possible, use Kerberos only
- 2. Implement Credential Guard (Windows 10+) to protect credentials in memory
- 3. Use Local Admin Password Solution (LAPS) for unique local admin passwords
- 4. Principle of least privilege: Don't use domain admin accounts for daily tasks
- 5. Monitor for suspicious authentication patterns (Event ID 4624, 4625, 4648)

Command-Line Account Management

Linux:

```
$ cat /etc/passwd | grep -v nologin
Lists users with login shells. Look for unexpected accounts.

$ sudo cat /etc/shadow | awk -F: '{print $1,$2}' | grep -v '!'
Shows users with password hashes (not locked). Empty hash = no password required!

$ lastlog
Shows last login time for all users. Inactive accounts should be disabled.

$ passwd -S username
Shows password status: locked, valid, expired.

$ sudo usermod -L username
Locks account by prepending '!' to password hash. Prevents password authentication.
```

Windows:

```
C:\> net user
Lists local user accounts. Look for unexpected admin accounts.

C:\> net localgroup Administrators
Shows members of local admin group. Should be minimal.

C:\> Get-LocalUser | Where-Object {$_.Enabled -eq $true}
PowerShell to list enabled local accounts. Disabled accounts can still be activated.

C:\> Get-ADUser -Filter * -Properties LastLogonDate | Where-Object {$_.LastLogonDate
-lt (Get-Date).AddDays(-90)}
Domain: Find accounts inactive for 90+ days. Should be disabled or deleted.
```

■ Under-Explored Perspectives: Authentication

1. Cached Credentials: Windows caches domain credentials locally for offline login. Even if domain is unreachable, last 10 users (default) can still log in. Cached creds are encrypted but extractable with admin access. Security implications are often missed.

2. Linux PAM Ordering Matters: PAM modules are evaluated in order. 'sufficient' means success stops processing. 'required' means failure is noted but processing continues. Misconfiguration can bypass authentication entirely.

3. Windows SID Reuse: Every user has a unique Security Identifier (SID). If you delete a user and create a new one with the same name, they get a different SID. Permissions tied to the old SID are lost. This is often confusing in ACL management.

4. SSH Key-Based Auth Nuances: SSH public key auth is more secure than passwords, but misconfigured `~/.ssh/authorized_keys` (wrong permissions, no `from=` restrictions) can be exploited. Plus, stolen private keys are often never rotated.

5. Password Hashing Iterations: Strong password hashes use many iterations (rounds) to slow brute-forcing. SHA-512 on Linux uses 5000 rounds by default. Higher is better but impacts login

performance. This tunability is rarely discussed.

Conclusion: System Security as a Foundation

Operating system security is the bedrock upon which all other security controls are built. You can have perfect firewalls, sophisticated intrusion detection, and advanced threat intelligence—but if your systems have weak file permissions, unnecessary services, or poor authentication, you will be compromised.

The four concepts covered—**process management and privilege escalation, file permissions and access control, service management and attack surface reduction, and authentication mechanisms**—are interconnected. An attacker exploits a vulnerable service (attack surface), gains initial access, exploits misconfigured file permissions to read sensitive data, abuses process privileges to escalate, and steals authentication credentials to persist and move laterally.

Understanding how Linux and Windows work at this level allows you to: design secure systems, audit existing configurations, investigate compromises, and most importantly, think like an attacker to anticipate their moves.

Key Takeaways

1. Every process has a privilege context. Understand UID/EUID, capabilities, tokens, and how privilege escalation works.
2. File permissions are the foundation of access control. Master Linux rwx and Windows ACLs. Audit for misconfigurations regularly.
3. Every running service is an attack vector. Minimize attack surface by disabling unnecessary services and hardening required ones.
4. Authentication is your first defense. Implement MFA, use strong password hashing, monitor for credential theft and abuse.
5. Defense in depth: These concepts work together. One weak layer compromises all others.

Your path forward: Practice these concepts in a lab environment. Spin up Linux and Windows VMs, deliberately misconfigure them, then practice exploitation and remediation. This hands-on experience is what will make these concepts stick and prepare you for real-world security engineering.